

PROBLEM 1

Problem Statement:

Given an integer N, check whether N is a Prime number or not using a user-defined function and display the result.

Input format:

A single integer N

Output format:

Print Prime if N is a prime number, otherwise print Not Prime

Sample Testcases:

Testcase1

Input: 29

Output: Prime

Testcase2

Input: 42

Output: Not Prime

Solution:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        long n = sc.nextLong();
        if(isPrime(n))
            System.out.print("Prime");
        else
            System.out.print("Not Prime");
    }
    static boolean isPrime(long n)
    {
        if(n < 2) return false; // 0 and 1 are not prime
        if(n % 2 == 0) return n == 2; // handle even numbers fast
        for(long i = 3; i * i <= n; i += 2)
        {
            if(n % i == 0)
                return false;
        }
        return true;
    }
}
```

Explanation of Logic:

A prime number has exactly two factors: 1 and itself. If N is less than 2, it is not prime. If N is even and greater than 2, it is not prime. Otherwise, check divisibility from 3 up to square root of N ($i*i \leq N$). If any divisor divides N exactly, it is not prime; otherwise, it is prime.

Example walkthrough with input 29:

Check divisors 3,5 (since $5*5 \leq 29$). None divide 29 exactly, so it is Prime.

PROBLEM 2

Problem Statement:

Given an integer N, check whether it is an Abundant number or not. An Abundant number is a number for which the sum of its proper divisors (excluding the number itself) is greater than the number.

Input format:

A single integer N

Output format:

Print Abundant if N is an abundant number, otherwise print Not Abundant

Sample Testcases:

Testcase1

Input: 12

Output: Abundant

Testcase2

Input: 15

Output: Not Abundant

Solution:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if(isAbundant(n))
            System.out.print("Abundant");
        else
            System.out.print("Not Abundant");
    }
    static boolean isAbundant(int n)
    {
        if(n <= 1) return false;
        int sum = 1; // 1 is a proper divisor for n>1
        for(int i = 2; i * i <= n; i++)
        {
            if(n % i == 0)
            {
                sum += i;
                if(i != n / i)
                    sum += n / i;
            }
        }
        return sum > n;
    }
}
```

Explanation of Logic:

Proper divisors are numbers less than N that divide N exactly. Start sum at 1, then check divisors from 2 up to square root of N. If i divides N, add both i and N/i (when they are different). After summing all proper divisors, compare the sum with N. If sum is greater than N, it is Abundant.

Example walkthrough with input 12:

Proper divisors are 1, 2, 3, 4, 6

Sum = 1+2+3+4+6 = 16

Since 16 > 12, output is Abundant.

PROBLEM 3

Problem Statement:

Given an integer N, check whether it is a Perfect number or not. A Perfect number is a number for which the sum of its proper divisors (excluding the number itself) is equal to the number.

Input format:

A single integer N

Output format:

Print Perfect if N is a perfect number, otherwise print Not Perfect

Sample Testcases:

Testcase1

Input: 6

Output: Perfect

Testcase2

Input: 10

Output: Not Perfect

Solution:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if(isPerfect(n))
            System.out.print("Perfect");
        else
            System.out.print("Not Perfect");
    }
    static boolean isPerfect(int n)
    {
        if(n <= 1) return false;
        int sum = 1; // 1 is a proper divisor for n>1
        for(int i = 2; i * i <= n; i++)
        {
            if(n % i == 0)
            {
                sum += i;
                if(i != n / i)
                    sum += n / i;
            }
        }
        return sum == n;
    }
}
```

Explanation of Logic:

A perfect number equals the sum of its proper divisors. Start sum as 1 for $n > 1$. Check divisors from 2 up to square root of N. When a divisor i divides N, add both i and N/i if they are different. After summing all proper divisors, compare sum with N. If equal, it is Perfect.

Example walkthrough with input 6:

Proper divisors are 1, 2, 3

Sum = $1+2+3 = 6$

Since sum equals the number, output is Perfect.

PROBLEM 4

Problem Statement:

Given an integer N, check whether it is a Harshad (Niven) number or not. A Harshad number is a number that is divisible by the sum of its digits.

Input format:

A single integer N

Output format:

Print Harshad if N is a Harshad number, otherwise print Not Harshad

Sample Testcases:

Testcase1

Input: 18

Output: Harshad

Testcase2

Input: 19

Output: Not Harshad

Solution:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if(isHarshad(n))
            System.out.print("Harshad");
        else
            System.out.print("Not Harshad");
    }
    static boolean isHarshad(int n)
    {
        if(n == 0) return false;
        int sum = digitSum(Math.abs(n));
        return sum != 0 && n % sum == 0;
    }
    static int digitSum(int x)
    {
        int s = 0;
        while(x > 0)
        {
            s += x % 10;
            x /= 10;
        }
        return s;
    }
}
```

Explanation of Logic:

Compute the sum of digits of N by repeatedly extracting digits using %10 and dividing by 10. Then check whether N is divisible by this digit sum. If divisible, it is a Harshad number.

Example walkthrough with input 18:

Digit sum = 1 + 8 = 9

18 % 9 = 0

Since it is divisible, output is Harshad.

PROBLEM 5

Problem Statement:

Given an integer N, check whether it is an Automorphic number or not using a user defined function. An Automorphic number is a number whose square ends with the same digits as the number itself.

Input format:

A single integer N

Output format:

Print Automorphic if N is an automorphic number, otherwise print Not Automorphic

Sample Testcases:

Testcase1

Input: 25

Output: Automorphic

Testcase2

Input: 13

Output: Not Automorphic

Solution:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        long n = sc.nextLong();
        if(isAutomorphic(n))
            System.out.print("Automorphic");
        else
            System.out.print("Not Automorphic");
    }
    static boolean isAutomorphic(long n)
    {
        if(n < 0) return false;
        long sq = n * n;
        long temp = n;
        while(temp > 0)
        {
            if((temp % 10) != (sq % 10))
                return false;
            temp /= 10;
            sq /= 10;
        }
        return true;
    }
}
```

Explanation of Logic:

Square the number. Then compare digits of the original number and its square from right to left. If all digits match until the original number becomes zero, the square ends with the same digits, so it is Automorphic.

Example walkthrough with input 25:

Square = 625

Compare from right: 5 matches 5, then 2 matches 2

All digits matched, so output is Automorphic.

PROBLEM 6

Problem Statement:

Given an integer N, check whether it is a Neon number or not. A Neon number is a number where the sum of digits of its square is equal to the number itself.

Input format:

A single integer N

Output format:

Print Neon if N is a neon number, otherwise print Not Neon

Sample Testcases:

Testcase1

Input: 9

Output: Neon

Testcase2

Input: 8

Output: Not Neon

Solution:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if(isNeon(n))
            System.out.print("Neon");
        else
            System.out.print("Not Neon");
    }
    static boolean isNeon(int n)
    {
        if(n < 0) return false;
        int sq = n * n;
        int sum = 0;
        while(sq > 0)
        {
            sum += sq % 10;
            sq /= 10;
        }
        return sum == n;
    }
}
```

Explanation of Logic:

First compute the square of the number. Then find the sum of digits of that square by repeatedly taking remainder with 10 and dividing by 10. If this digit sum equals the original number, it is a Neon number.

Example walkthrough with input 9:

Square = 81

Digit sum = 8 + 1 = 9

Since digit sum equals the number, output is Neon.

PROBLEM 7

Problem Statement:

Given an integer N, check whether it is a Duck number or not. A Duck number contains at least one zero in its digits but does not start with zero.

Input format:

A single integer N

Output format:

Print Duck if N is a duck number, otherwise print Not Duck

Sample Testcases:

Testcase1

Input: 1023

Output: Duck

Testcase2

Input: 1234

Output: Not Duck

Solution:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        if(isDuck(s))
            System.out.print("Duck");
        else
            System.out.print("Not Duck");
    }
    static boolean isDuck(String s)
    {
        if(s.length() == 0) return false;
        if(s.charAt(0) == '0') return false; // leading zero not allowed
        for(int i = 1; i < s.length(); i++)
        {
            if(s.charAt(i) == '0')
                return true;
        }
        return false;
    }
}
```

Explanation of Logic:

A duck number must contain at least one zero but not as the first digit. Read the input as string to inspect digits directly. If the first character is '0', it is not duck. Otherwise scan remaining digits and if any digit is '0', it is duck.

Example walkthrough with input 1023:

First digit is 1 (valid), digits contain 0 at position 2, so it is Duck.

PROBLEM 8

Problem Statement:

Given an integer N, check whether it is a Spy number or not. A Spy number is a number in which the sum of its digits equals the product of its digits.

Input format:

A single integer N

Output format:

Print Spy if N is a spy number, otherwise print Not Spy

Sample Testcases:

Testcase1

Input: 1124

Output: Spy

Testcase2

Input: 123

Output: Not Spy

Solution:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if(isSpy(n))
            System.out.print("Spy");
        else
            System.out.print("Not Spy");
    }
    static boolean isSpy(int n)
    {
        n = Math.abs(n);
        int sum = 0;
        int prod = 1;
        if(n == 0) return false;
        while(n > 0)
        {
            int d = n % 10;
            sum += d;
            prod *= d;
            n /= 10;
        }
        return sum == prod;
    }
}
```

Explanation of Logic:

Extract each digit using remainder with 10. Add digits to a running sum and multiply them into a running product. After processing all digits, compare sum and product. If they are equal, the number is a Spy number.

Example walkthrough with input 1124:

Digits are 1,1,2,4

Sum = 1+1+2+4 = 8

Product = 1*1*2*4 = 8

Since sum equals product, output is Spy.

PROBLEM 9

Problem Statement:

Given an integer N, check whether it is a Disarium number or not. A Disarium number is a number in which the sum of its digits powered with their respective positions (starting from 1 from left) equals the number itself.

Input format:

A single integer N

Output format:

Print Disarium if N is a Disarium number, otherwise print Not Disarium

Sample Testcases:

Testcase1

Input: 135

Output: Disarium

Testcase2

Input: 123

Output: Not Disarium

Solution:

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if(isDisarium(n))
            System.out.print("Disarium");
        else
            System.out.print("Not Disarium");
    }
    static boolean isDisarium(int n)
    {
        if(n < 0) return false;
        int digits = countDigits(n);
        int temp = n;
        int sum = 0;
        while(temp > 0)
        {
            int d = temp % 10;
            sum += power(d, digits);
            digits--;
            temp /= 10;
        }
        return sum == n;
    }
    static int countDigits(int x)
    {
        if(x == 0) return 1;
        int c = 0;
        while(x > 0)
        {
            c++;
            x /= 10;
        }
    }
}
```

```

        return c;
    }
    static int power(int base, int exp)
    {
        int p = 1;
        for(int i = 0; i < exp; i++)
            p *= base;
        return p;
    }
}

```

Explanation of Logic:

First count the number of digits. Then extract digits from right to left. The rightmost digit gets the highest position power, decreasing each step. Add $\text{digit}^{\text{position}}$ to a running sum. If the final sum equals the original number, it is Disarium.

Example walkthrough with input 135:

Digits count = 3

From right: $5^3 + 3^2 + 1^1 = 125 + 9 + 1 = 135$

Since sum equals the number, output is Disarium.

PROBLEM 10

Problem Statement:

Given an integer N, check whether it is a Happy number or not. A Happy number is obtained by repeatedly replacing the number with the sum of squares of its digits until it becomes 1. If the process reaches 1, it is Happy; if it falls into a loop not including 1, it is Not Happy.

Input format:

A single integer N

Output format:

Print Happy if N is a happy number, otherwise print Not Happy

Sample Testcases:

Testcase1

Input: 19

Output: Happy

Testcase2

Input: 20

Output: Not Happy

Solution:

```

import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if(isHappy(n))
            System.out.print("Happy");
        else
            System.out.print("Not Happy");
    }
    static boolean isHappy(int n)
    {

```

```

        if(n <= 0) return false;
        int slow = n;
        int fast = n;
        do
        {
            slow = sumSquareDigits(slow);
            fast = sumSquareDigits(sumSquareDigits(fast));
        }
        while(slow != fast);
        return slow == 1;
    }
    static int sumSquareDigits(int x)
    {
        int s = 0;
        while(x > 0)
        {
            int d = x % 10;
            s += d * d;
            x /= 10;
        }
        return s;
    }
}

```

Explanation of Logic:

Replace the number by the sum of squares of its digits repeatedly. Happy numbers end at 1, while others fall into a repeating cycle. Use two runners (slow and fast) where slow moves one step and fast moves two steps each round. If they meet at 1, the number is Happy; otherwise, it is Not Happy.

Example walkthrough with input 19:

19 -> 82 -> 68 -> 100 -> 1

Since the sequence reaches 1, output is Happy.

PROBLEM 11

Problem Statement:

Given an integer N, check whether it is a Kaprekar number or not. A Kaprekar number is a number whose square can be split into two parts that add up to the original number (right part must have the same number of digits as N).

Input format:

A single integer N

Output format:

Print Kaprekar if N is a Kaprekar number, otherwise print Not Kaprekar

Sample Testcases:

Testcase1

Input: 45

Output: Kaprekar

Testcase2

Input: 46

Output: Not Kaprekar

Solution:

```

import java.util.*;
public class Main
{

```

```

public static void main(String[] args)
{
    Scanner sc = new Scanner(System.in);
    long n = sc.nextLong();
    if(isKaprekar(n))
        System.out.print("Kaprekar");
    else
        System.out.print("Not Kaprekar");
}
static boolean isKaprekar(long n)
{
    if(n < 1) return false;
    if(n == 1) return true;
    long sq = n * n;
    int d = countDigits(n);
    long p = pow10(d);
    long right = sq % p;
    long left = sq / p;
    return right > 0 && (left + right == n);
}
static int countDigits(long x)
{
    int c = 0;
    while(x > 0)
    {
        c++;
        x /= 10;
    }
    return c;
}
static long pow10(int e)
{
    long p = 1;
    for(int i = 0; i < e; i++)
        p *= 10;
    return p;
}
}

```

Explanation of Logic:

Square the number N. Count digits d in N and compute 10^d . Split the square into two parts: $\text{right} = \text{square} \% 10^d$ and $\text{left} = \text{square} / 10^d$. If right part is not zero and $\text{left} + \text{right}$ equals N, then it is a Kaprekar number.

Example walkthrough with input 45:

Square = 2025, digits in 45 = 2, $p = 100$

Right = $2025 \% 100 = 25$

Left = $2025 / 100 = 20$

$20 + 25 = 45$, so output is Kaprekar.